

Ускорение инференса UNet модели

Никитюк Илья, Потапенко Антон, Тюменцева Ирина

27 апреля 2026

1 Описание проекта

Описание проекта можно найти на [странице описания проектов](#) курса ШАД [Эффективные модели ML и архитектуры нейросетей](#). И в репозитории [проекта](#).

Цель проекта - ускорение инференса UNet-модели. В качестве бейзлайна рассматривается инференс на PyTorch с вычислениями в формате fp16. Для ускорения инференса относительно бейзлайна применяются: PyTorch torch.compile (JIT-компиляция графа вычислений); компилятор TVM (аппаратно-независимая оптимизация через Relax и TIR), квантизация (снижение точности весов и активаций), спарсификация: pruning / 2:4 semi-structured (разреживание весовой матрицы). Вклад квантизации и прунинга оценивается отдельно посредством профилирования, измерения качества, latency и пропускной способности на инференсе.

В качестве итогов проекта ожидается:

1. Сравнительная таблица latency и throughput для всех рассмотренных конфигураций.
2. Измерение времени инференса после применения оптимизаций с контролем качества (метрики точности) относительно fp16-референса.
3. Публичный git-репозиторий с воспроизводимыми скриптами бенчмаркинга, включая requirements.txt/pyproject.toml и/или Dockerfile (либо ссылка на готовый Docker-образ).
4. Документация в формате Markdown с подробным описанием примененных методов ускорения и полученных результатов.
5. Итоговая презентация, содержащая ключевые выводы по эффективности каждого из подходов.

2 Планирование экспериментов

План экспериментов:

1. Построить baseline модели `Unet`
2. Провести эксперименты с использованием компиляторных оптимизаций
 - `torch.compile`
 - `torch.compile` с параметрами `max_autotune` и `default`
 - TVM compiler
3. Провести эксперименты с использованием оптимизаций
 - Квантизация
 - Спарсификация
4. Провести эксперименты с использованием комбинированных методов оптимизаций
 - `torch.compile` + квантизация
 - `torch.compile` + спарсификация

3 Обучение baseline модели

Модель Unet получена дообучением предобученной модели из [segmentation models pytorch](#) на COCO-датасете. Обучение проходило с точностью bfloat16. Веса энкодера в процессе обучения не менялись (были заморожены). Оценка качества полученной модели проводилась на валидационном COCO-датасете.

Воспроизвести обучение можно с помощью [train_COCO.ipynb](#)

Параметры модели UNet:

- энкодер: ResNet101
- датасет предобучения энкодера: ImageNet
- обучаемые веса: декодер
- всего параметров: 51.524.833, обучаемых из них: 9.024.673

Датасет:

- train: [изображения](#), [аннотации](#)
- validation: [изображения](#), [аннотации](#)

Параметры обучения:

- Loss: CrossEntropyLoss
- Optimizer: AdamW
- learning rate: 0.001 + scheduler
- Количество эпох: 20

Программное обеспечение:

- python: 3.13.7
- cuda: 13.2
- torch: 2.11

Аппаратное обеспечение:

- CPU: Ryzen 7700x
- GPU: Nvidia 4070 ti Super

Процесс обучения и результаты обучения представлены на изображениях далее.

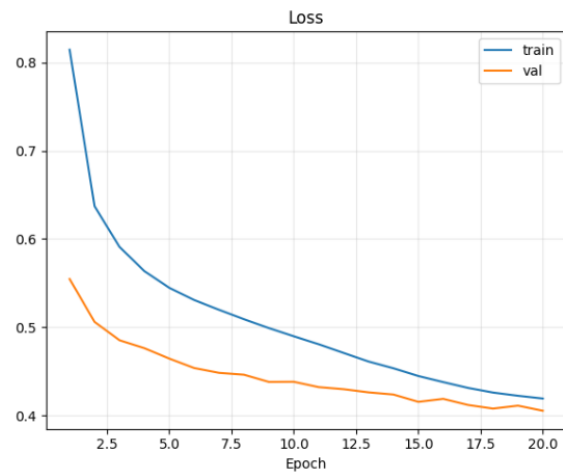


Рис. 1: График изменения функции потерь

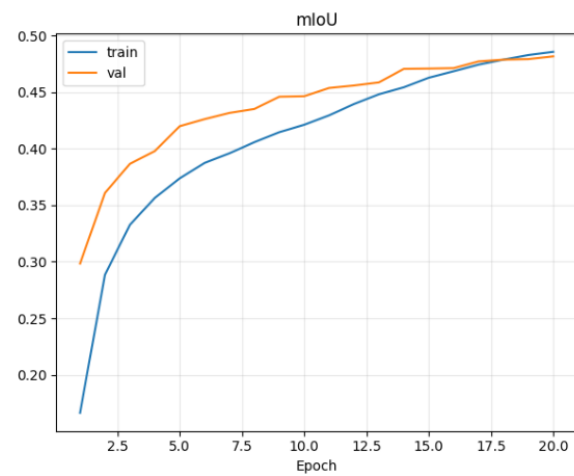


Рис. 2: График изменения метрики качества mIoU

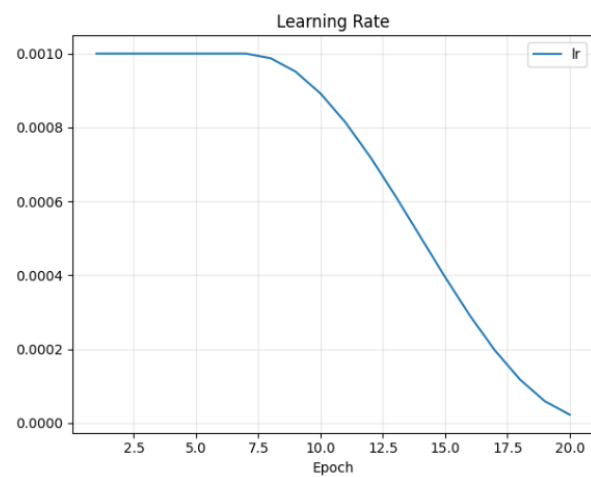


Рис. 3: График изменения шага обучения

4 Эксперименты с `compile` оптимизациями

4.1 Цель эксперимента

Целью эксперимента являлось сравнение нескольких способов выполнения инференса модели сегментации UNet с энкодером ResNet101 на GPU:

- исходная реализация в PyTorch;
- `torch.compile` в стандартном режиме;
- `torch.compile` с оптимизацией;
- TVM Relax.

Сравнение проводилось по трем основным критериям (на A100):

- средняя задержка инференса (Latency);
- пропускная способность (Throughput) при различных размерах батча;
- значение основной метрики качества (mIoU).

4.2 Полученные результаты

4.2.1 Задержка инференса и качество

Таблица 1: Сравнение по задержке и качеству

Метод	Тип данных	Latency, ms	mIoU, %
baseline	float16	19.128	48.1880
<code>torch.compile</code> (default)	float16	5.825	48.1914
<code>torch.compile</code> (tuned)	float16	2.520	48.1885
TVM Relax	float32	26.583	48.1789

Из таблицы видно, что:

- наименьшую задержку показал вариант `torch.compile` (tuned);
- вариант `torch.compile` (default) также значительно превосходит baseline;
- TVM Relax в текущей реализации оказался медленнее baseline;
- значения mIoU во всех вариантах практически совпадают, то есть ускорение или замедление не сопровождалось заметной деградацией качества.

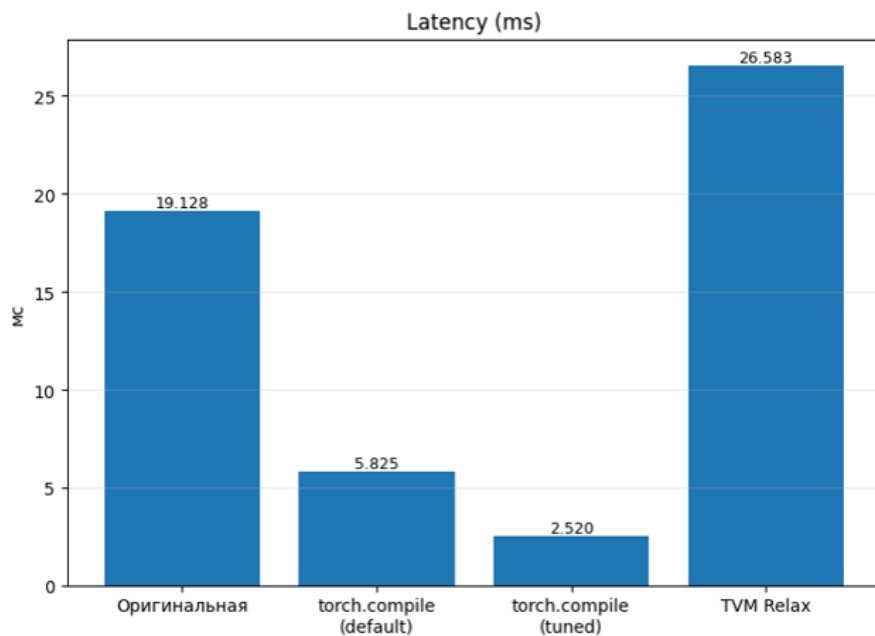


Рис. 6: Latency

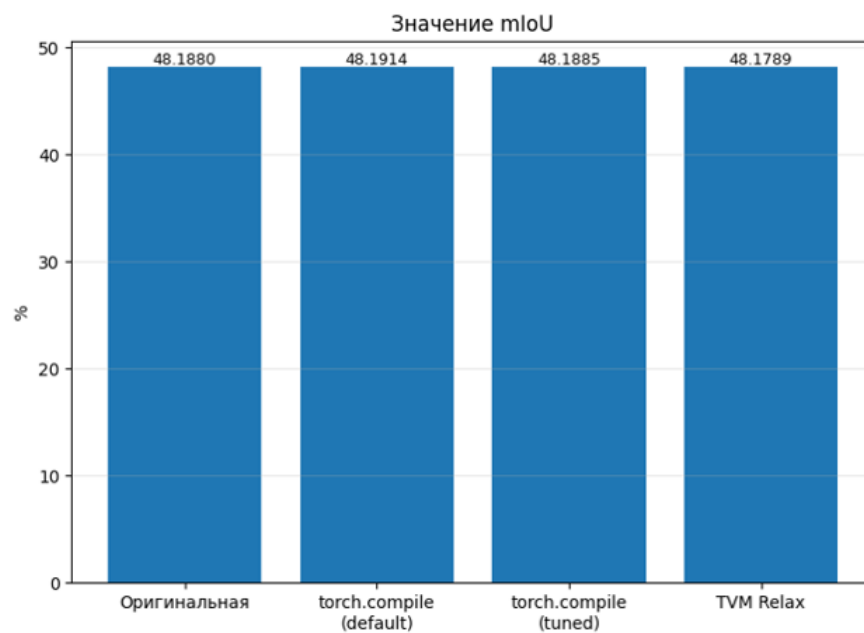


Рис. 7: mIoU

Таблица 2: Пропускная способность, изображений в секунду

Метод	bs=1	bs=2	bs=4	bs=8	bs=16	bs=32	bs=64
baseline	51.03	110.27	227.87	398.15	584.85	608.70	621.20
torch.compile (default)	175.54	299.89	547.20	874.00	972.51	1014.05	1057.83
torch.compile (tuned)	387.93	676.09	880.77	980.28	1075.94	1171.47	1179.13
TVM Relax	37.44	55.28	73.19	79.16	82.97	86.65	87.33

4.2.2 Пропускная способность при различных размерах батча

По пропускной способности наблюдается та же тенденция, что и по задержке:

- `torch.compile (tuned)` является лучшим вариантом практически на всех размерах батча;
- `torch.compile (default)` также существенно превосходит исходную реализацию;
- TVM Relax показывает значительно более низкую пропускную способность, чем все PyTorch-варианты.

4.2.3 Количественный анализ ускорения

Относительно исходной модели ускорение по latency составляет:

$$S_{\text{default}} = \frac{19.596}{5.697} \approx 3.44,$$

$$S_{\text{tuned}} = \frac{19.596}{2.578} \approx 7.60,$$

$$S_{\text{TVM}} = \frac{19.596}{26.708} \approx 0.73.$$

Следовательно:

- `torch.compile (default)` ускоряет инференс примерно в 3.4 раза;
- `torch.compile (tuned)` ускоряет инференс примерно в 7.6 раза;
- TVM Relax в данной конфигурации не ускоряет вычисления, а, наоборот, замедляет их.

Если рассматривать максимальную пропускную способность на `batch_size = 64`, то:

- baseline: 621.20 изображений/с;
- `torch.compile (default)`: 1057.83 изображений/с;
- `torch.compile (tuned)`: 1179.13 изображений/с;
- TVM Relax: 87.33 изображений/с.

4.3 Что удалось решить по итогам разработки

В результате проведённой работы удалось:

1. Успешно запустить baseline и `torch.compile`;
2. Собрать TVM с поддержкой CUDA и построить TVM-пайплайн;
3. Выполнить измерение latency и throughput на наборах батчей 1, 2, 4, 8, 16, 32, 64;
4. Убедиться, что значения mIoU остаются практически идентичными во всех вариантах.

4.4 Анализ полученных результатов

По результатам экспериментов можно сделать следующие выводы:

1. Для рассматриваемой модели `Unet` и в текущей экспериментальной конфигурации наилучшие результаты по скорости показал `torch.compile`, особенно в `tuned`-режиме.
2. Вариант `torch.compile (tuned)` обеспечил минимальную задержку инференса и максимальную пропускную способность при всех исследованных размерах батча.
3. TVM Relax в текущей реализации не дал выигрыша по скорости: он уступил как `baseline`, так и обоим вариантам `torch.compile`.
4. При этом качество сегментации во всех вариантах осталось практически одинаковым, так как значения `mIoU` отличаются лишь на уровне тысячных и десятитысячных долей процента. Поэтому, основное различие между подходами определяется не качеством инференса, а исключительно эффективностью исполнения.

* Стоит отметить, что при реализации TVM-пайплайна возникли проблемы с корректной работой данного компилятора:

- Запуск TVM удалось произвести на `float32`, что в свою очередь увеличивает задержку инференса;
- Также часть возникших конфликтов между CUDA и TVM не до конца удалось решить, поэтому некоторые операции расчетов модели не были оптимизированы. В связи с этим, были получены не самые оптимальные результаты.

5 Эксперименты с aware training оптимизациями

5.1 LSQ квантизация с инференсом F.conv2d

Реализация lsq состоит в симуляции квантизации "на лету"(fake quantization) с использованием стандартного `F.conv2d` в PyTorch.

Код предлагает два режима: обучающий класс `QACConv2d` и инференсный класс `QACConv2dInference`. В инференсном режиме веса и масштабы квантизации фиксируются:

```
with torch.no_grad():
    s_w = original_layer.weight_scale.clamp(min=1e-8)
    w_q = torch.clamp(torch.round(original_layer.weight / s_w), self.Qn, self.Qp) * s_w
    self.weight.copy_(w_q)
```

Квантизованные веса сохраняются непосредственно в `self.weight` как обычные тензоры типа `float32`. При инференсе выполняется:

```
s_x = self.x_scale.clamp(min=1e-8)
x = torch.clamp(torch.round(x / s_x), self.Qn, self.Qp) * s_x
return super().forward(x)
```

Профилирование показывает, что на один свёрточный слой в инференсном режиме приходится следующая последовательность операций на CPU:

1. `aten::pad` – операция статического паддинга (46-60 мкс)
2. `aten::constant_pad_nd` – реализация паддинга (44-57 мкс)
3. `aten::empty + aten::fill_` – выделение и заполнение тензора (10-20 мкс)
4. `aten::narrow + aten::slice + aten::as_strided` – операции нарезки (6-10 мкс)
5. `aten::copy_` – копирование данных (13-14 мкс)
6. `aten::clamp` – для ограничения значений после деления (16-18 мкс)
7. `aten::round` – округление после масштабирования (12-14 мкс)
8. `aten::div` и `aten::mul` – масштабирование (14-16 мкс)
9. `aten::cudnn_convolution` – собственно свёртка (41-75 мкс)

Для каждого свёрточного слоя выполняется до 15-20 отдельных CPU-операций перед вызовом `cudnn_convolution`. Это создаёт Python-оверхед на малых батчах.

Также из трейса 9 видно, что каждая операция (`clamp`, `round`, `div`, `mul`) порождает отдельный вызов CUDA-ядра.

При использовании `F.conv2d` с квантизованными весами, хранящимися в `float32`, реальное перемножение происходит в арифметике с плавающей точкой. Это означает, что:

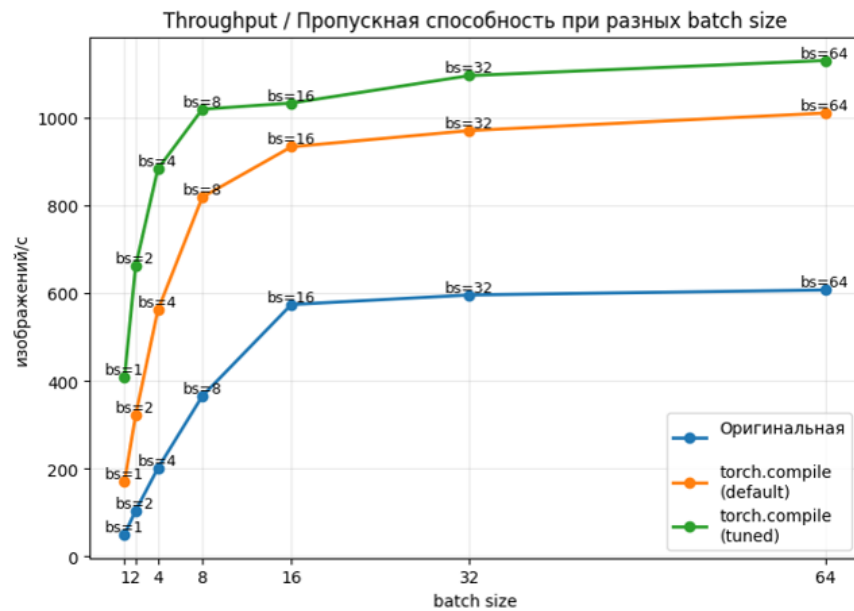


Рис. 8: Пропускная способность

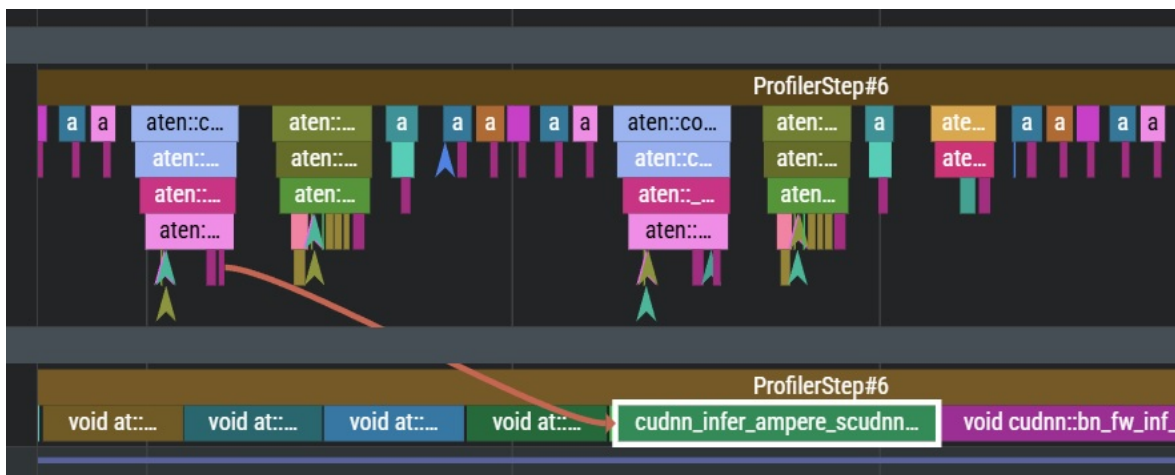


Рис. 9: Скрин визуализации профилирования инференса модели с F.conv2d

- Размер значений равен 23 битам (float32 / float16)
- Нет реального использования целочисленной арифметики
- Энергопотребление и пропускная способность памяти соответствуют FP32 / FP16

То есть данная реализация не даёт аппаратных преимуществ. Для реального ускорения инференса с LSQ надо:

1. **Имплементация целочисленного инференса**
2. **Слияние операций:** объединение `scale` $\rightarrow$ `round` $\rightarrow$ `clamp` $\rightarrow$ `scale` в одно ядро

Имплементация описана в разделе ниже.

5.2 LSQ квантизация с инференсом на Triton-ядре

Основным недостатком реализации LSQ со стандартным `F.conv2d` является арифметика с плавающей точкой: операции масштабирования, округления и клиппинга выполняются в `fp32` / `fp16`, а целочисленная (`int8`) арифметика не используется. Это приводит к высоким накладным расходам на CPU (до 15-20 отдельных операций на слой) и не позволяет воспользоваться преимуществами целочисленных тензорных ядер.

Для преодоления этих ограничений было разработано кастомное Triton-ядро, реализующее аппаратно-эффективный инференс квантизованной LSQ-свертки [10](#).

Загрузка параметров квантизации. Параметры квантизации (масштаб активаций s_x , комбинированный масштаб $s_w \cdot s_x$, границы квантования Q_n и Q_p) загружаются из глобальной памяти в регистры. Это позволяет использовать единые значения для всего блока вычислений.

```
x_scale = tl.load(x_scale_ptr)
combined_scale = tl.load(combined_scale_ptr)
qn, qp = tl.load(qn_ptr), tl.load(qp_ptr)
```

Квантизация активаций "на лету". Для каждого элемента входного тензора x выполняется операция $\text{round}(x/s_x)$, после чего результат клипсируется в диапазон $[Q_n, Q_p]$ и приводится к типу `int8`.

```
a = tl.load(x_ptr + off_x, mask=mask_x, other=0.0)
a = tl.math.round(a / x_scale)
a = tl.maximum(a, qn)
a = tl.minimum(a, qp)
a = a.to(tl.int8)
```

Загрузка `int8` весов. Квантизованные веса w хранятся в формате `int8` и загружаются из глобальной памяти.

```
b = tl.load(w_ptr + off_w, mask=mask_w, other=0).to(tl.int8)
```

Целочисленное матричное умножение через Tensor Cores. Операция `accumulator += a · b` выполняется с использованием инструкций тензорных ядер через функцию `tl.dot`.

```
accumulator += tl.dot(a, b)
```

Де-квантизация и добавление bias. Накопленный целочисленный результат преобразуется в целевой тип данных (fp32, fp16 или bf16), умножается на комбинированный масштаб $s_w \cdot s_x$ и складывается со смещением *bias*.

```
out = accumulator.to(OUTPUT_TYPE) * combined_scale + bias_vals
tl.store(out_ptr + off_out, out, mask=mask_out)
```

Сравнение результатов профилирования (рис. 10) с трейсом для `F.conv2d` (рис. 9) показывает упрощение графа вычислений:

- Нет отдельных вызовов `aten::pad`, `aten::narrow`, `aten::copy_`.
- Все операции масштабирования, округления и клиппинга инкапсулированы внутри ядра `lsq_conv_kernel_g1`.
- Промежуточные вызовы CUDA-ядер для элементарных операций (`clamp`, `round`, `div`, `mul`) — заменены на скалярные операции внутри Triton-ядра.
- Вызов `cudnn_convolution` заменен на вызов целевого Triton-ядра, которое напрямую использует целочисленные тензорные ядра.

С точки зрения производительности, на уровне отдельного ядра Triton-реализация позволяет загрузить Tensor Cores для int8-вычислений, однако конкурировать с высокооптимизированными библиотеками (cuDNN, cuBLAS) не получилось. Достигнутая в текущей реализации производительность (конечная задержка, пропускная способность) представлена в разделе “Результаты” и показывает, что дальнейшая оптимизация памяти и блочной структуры является необходимым следующим шагом.

5.3 Неструктурный прунинг

В эксперименте использовался `torch.nn.utils.prune.l1_unstructured`. Этот метод делает неструктурный magnitude pruning: внутри выбранного слоя он находит веса с наименьшим абсолютным значением и зануляет заданную долю таких весов. То есть удаляются не каналы и не фильтры целиком, а отдельные элементы тензора весов. Фактически для слоя создаётся binary mask.

Проверялись крупные слои `Conv2d`. Для каждого слоя отдельно оценивалась деградация mIoU при разных значениях `amount`, затем был собран общий pruning plan.

Итоговая глобальная доля нулей составляла порядка 20% весов `Conv2d/Linear` слоёв. При этом качество почти не просело: mIoU изменился с 0.4819 до 0.4804.

Итог: модель действительно имеет параметрическую избыточность. Но ускорения в обычном dense-инференсе почти не будет, потому что форма слоёв не меняется, а dense `Conv2d` всё равно считает нули.

5.4 Структурный прунинг

В эксперименте использовался инструментарий `torch-pruning`. При удалении канала из одного свёрточного слоя `torch-pruning` корректирует соответствующие входные каналы следующего слоя.

То есть физически удалялись каналы в выбранных `Conv2d`-слоях.

После самого прунинга модель почти полностью сломалась: mIoU изменился с 0.4819 до 0.0100.

После `fine-tune` качество частично восстановилось: mIoU поднялся до 0.4608.

Итог: это самый полезный вариант для реального ускорения, потому что меняется архитектура модели и уменьшается число операций.

5.5 2:4 semi-structured sparsity

Для текущей модели 2:4 не выглядит перспективным методом.

Причина: модель почти полностью состоит из `Conv2d`, а 2:4 semi-structured sparsity в стандартном PyTorch-пайплайне в первую очередь полезна для sparse GEMM / `Linear`-слоёв.

Для этой U-Net/ResNet101: `Conv2d + 2:4 mask` $\approx$ dense `Conv2d` с нулями.

Итог: без специализированного backend или конвертации под sparse convolution реального ускорения ждать не стоит.

5.6 Вывод

Лучший практический результат в нашем случае — это structured pruning + fine-tune:

- минус 35.76% параметров;
- минус 34.26% MACs (Multiply–Accumulate operations - операции вида $a*b+c$);
- mIoU просел примерно на 2.1 процентного пункта после fine-tune.

Неструктурный прунинг показал, что модель избыточна, но сам по себе не даёт нормального ускорения.

2:4 для этой архитектуры почти бесполезен, потому что основная нагрузка приходится в `Conv2d`, а не в `Linear`/GEMM.

6 Эксперименты с комбинацией оптимизаций

6.1 LSQ квантизация с torch.compile

В рамках данного раздела исследуется совместное применение двух техник ускорения: LSQ-квантизации (Learned Step Size Quantization) и JIT-компиляции `torch.compile`. Каждая из этих техник по отдельности показала свою эффективность: LSQ позволяет снизить точность вычислений с сохранением качества за счёт обучения шага квантования, а `torch.compile` оптимизирует граф вычислений, выполняя слияние операций, устранение накладных расходов Python и автоподбор параметров ядер.

6.1.1 Torch.compile и F.conv2d

В данном эксперименте исследовалось совместное применение LSQ-квантизации (через стандартную функцию `F.conv2d`) и JIT-компиляции `torch.compile`. `Torch.compile` применялся в двух режимах: `default` и `max-autotune (tuned)`.

Результаты эксперимента представлены на рисунках 13, 11 и 12

Исходная `bf16`-модель без компиляции показала задержку 29.25 мс — это медленнее `fp16-baseline` (19.60 мс). После компиляции с `torch.compile (default)`, latency сократилось до 5.23 мс (ускорение в 5.6 раза относительно `bf16` без компиляции). `Torch.compile (auto-tuned)` достиг задержки 4.31 мс (ускорение в 6.8 раза).

Пропускная способность выросла с 51 img/s до 880 img/s.

Ключевая проблема данного эксперимента — снижение качества. Значение `mIoU` упало с 45.24 до 0.82. Есть предположение, что данная потеря качества связана с багом в реализации класса свёртки для инференса. Проверить гипотезу и устранить эту ошибку в рамках текущего проекта не получилось, т.к. бага была обнаружена слишком поздно и ставки делались на Triton-ядро + компиляцию.

6.1.2 Torch.compile и triton-ядро

В данном эксперименте исследовалась комбинация LSQ-квантизации (с собственным triton-ядром для сверток) и JIT-компиляции `torch.compile`. Triton-ядро было разработано для выполнения целочисленных `int8` операций квантизованной свертки. `torch.compile` применялся для снижения оверхеда Python, fusion ядер и других оптимизаций общей производительности.

Результаты эксперимента представлены на рисунках 14, 15 и 16

Важным результатом является сохранение точности работы модели для комбинации `torch.compile` и Triton-ядра, достигнутое на этапе LSQ-квантизации с Triton (`mIoU` = 47.50), что на 2.26 выше, чем у исходного `fp16-baseline`.

Влияние `torch.compile` на производительность kernels, написанного вручную на Triton, получилось положительным - `torch.compile` существенно ускорил выполнение: задержка сократилась с 40.00 мс до 17.3-17.8 мс, что соответствует ускорению более чем в 2.3 раза. Если сравнивать с другими экспериментами (не квантизация), то latency даже после компиляции (17.33 мс) хуже, чем у `torch.compile (max-autotune)` для `fp16` модели (2.578 мс). Пропускная способность (205 img/s) также уступает бейзлайну.

`Torch.compile` эффективно "склеивает" операции входа/выхода тензоров до и после Triton-ядра, при этом не может переписать алгоритм ядра (функцию `@triton.jit`). По причине того, что ядро пока неоптимально, возникает улучшение с 40 мс до 17 мс, но не до 2.5 мс. Для достижения максимальной производительности требуется дальнейшая оптимизация самого Triton-кernels (настройка BLOCK-размеров, использование Tensor Cores через `tt.dot`).

6.2 Структурный прунинг с torch.compile

К модели после прунинга применялась JIT-компиляция `torch.compile`. Результаты экспериментов представлены на рисунках 17, 18 и 19.

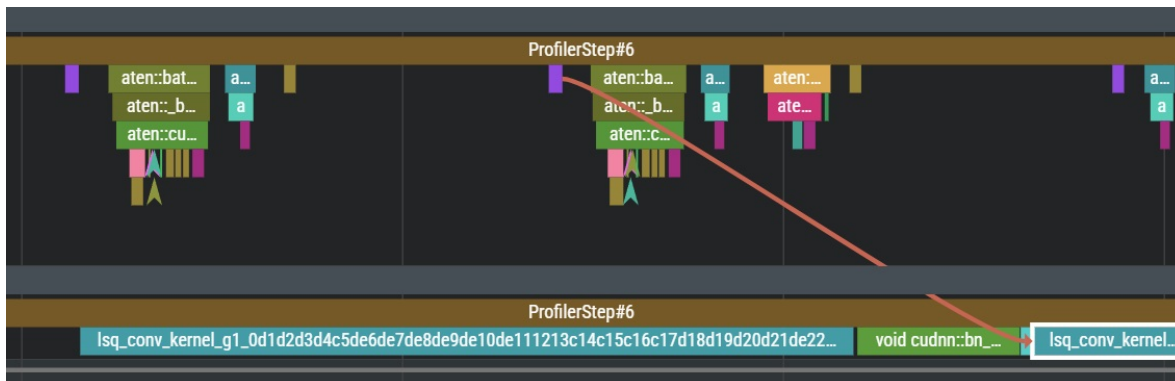


Рис. 10: Скрин визуализации профилирования инференса модели с Triton-ядром

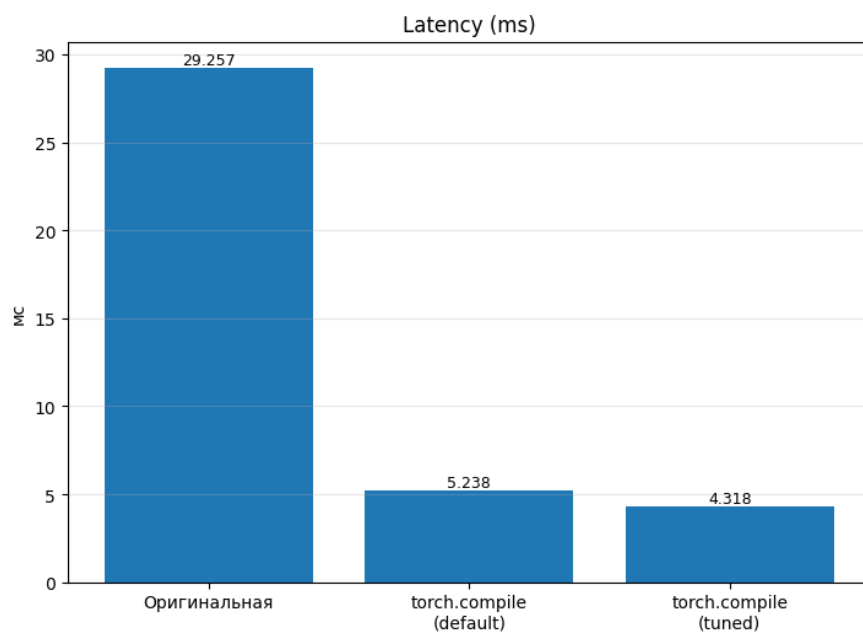


Рис. 11: График изменения latency для инференса с F.conv2d после компиляции

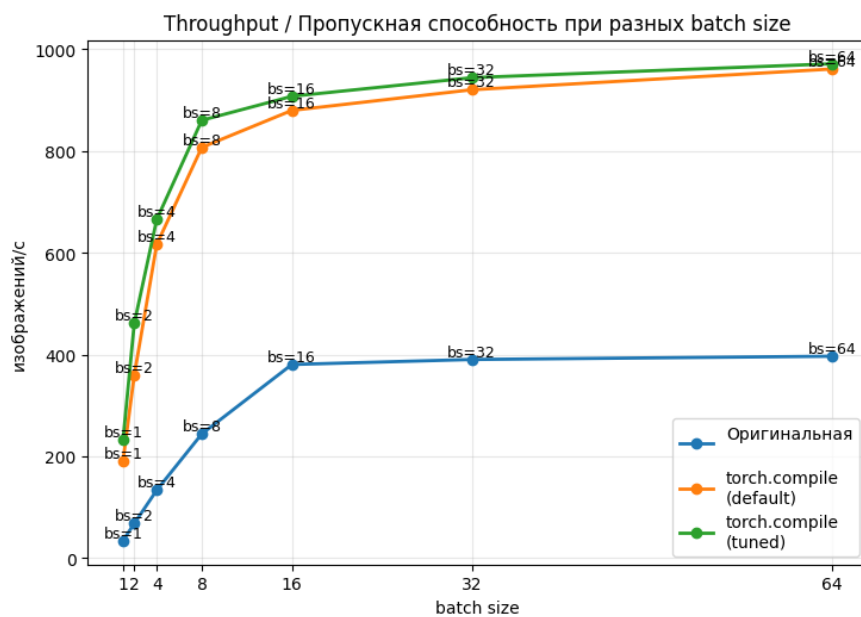


Рис. 12: График изменения пропускной способности для инференса с F.conv2d после компиляции

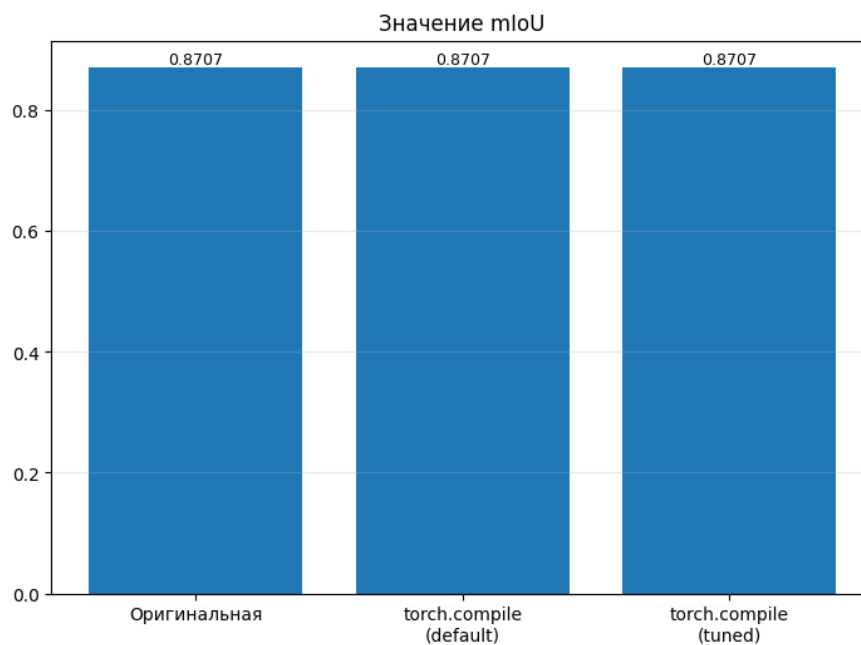


Рис. 13: График изменения качества для инференса с F.conv2d после компиляции

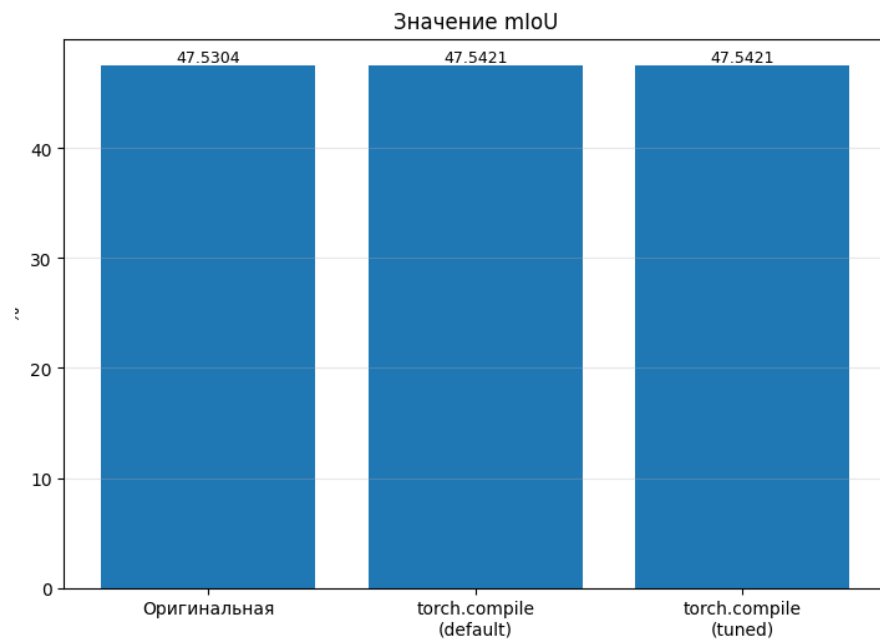


Рис. 14: График изменения качества для инференса на triton-ядре после компиляции

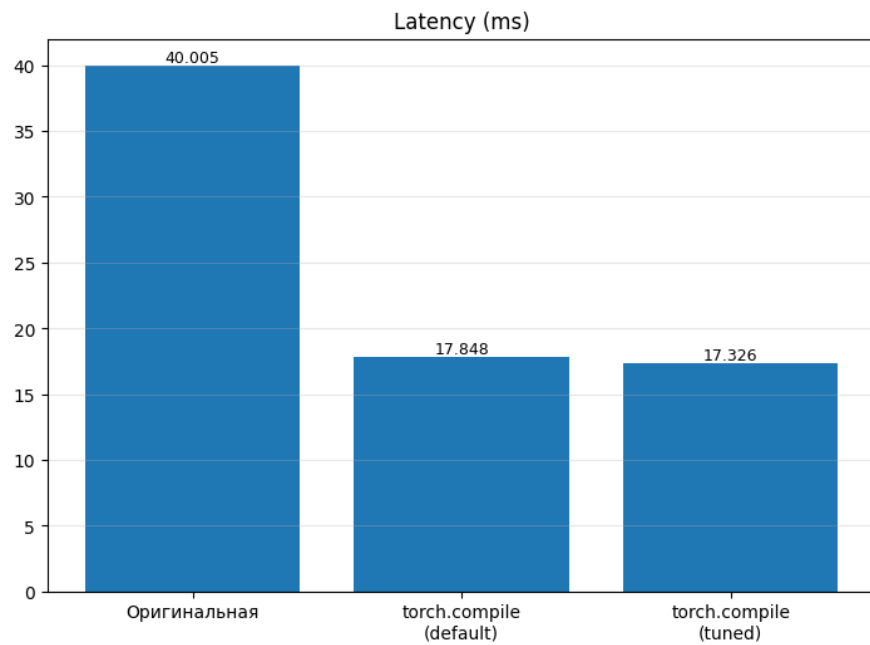


Рис. 15: График изменения latency для инференса на triton-ядре после компиляции

По результатам экспериментов можно сделать вывод, что комбинация прунинга и `torch.compile` позволяет достичь существенного прироста производительности. В режиме `max-autotune` удалось добиться ускорения в 8.5 раза по сравнению с моделью после прунинга без компиляции (снижение latency с 19.60 мс до 2.32 мс). Качество модели не изменилось по отношению к бейзлайну.

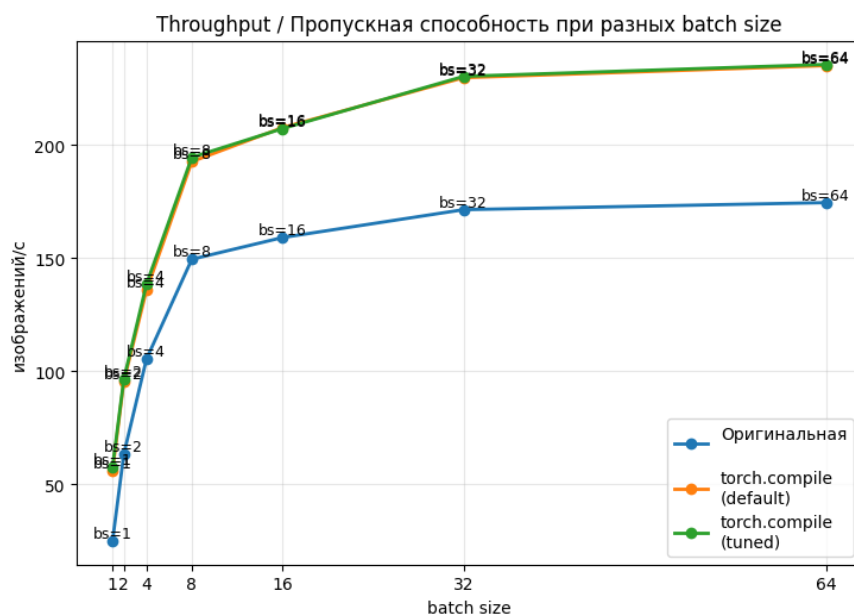


Рис. 16: График изменения пропускной способности для инференса на triton-ядре после компиляции

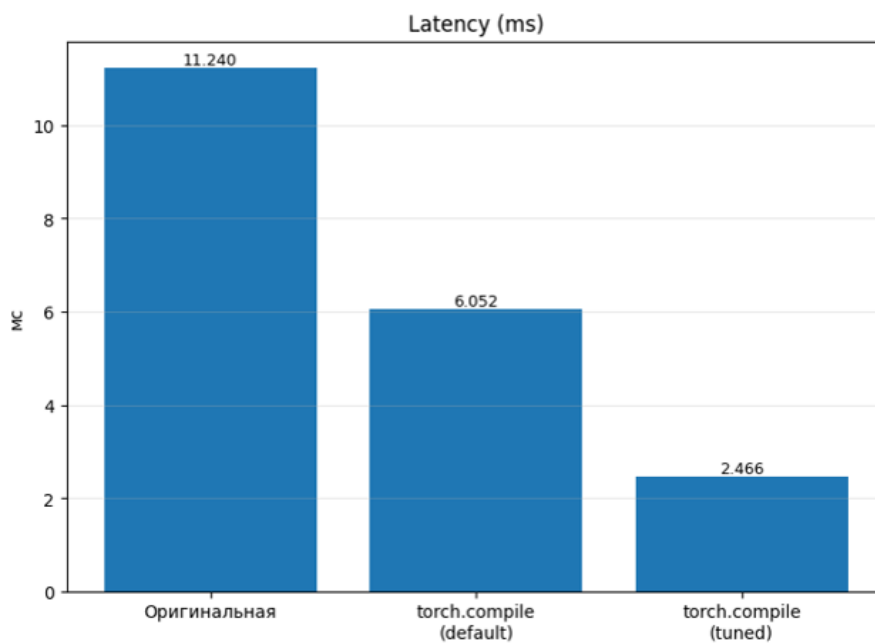


Рис. 17: График изменения latency для модели после прунинга и компиляции

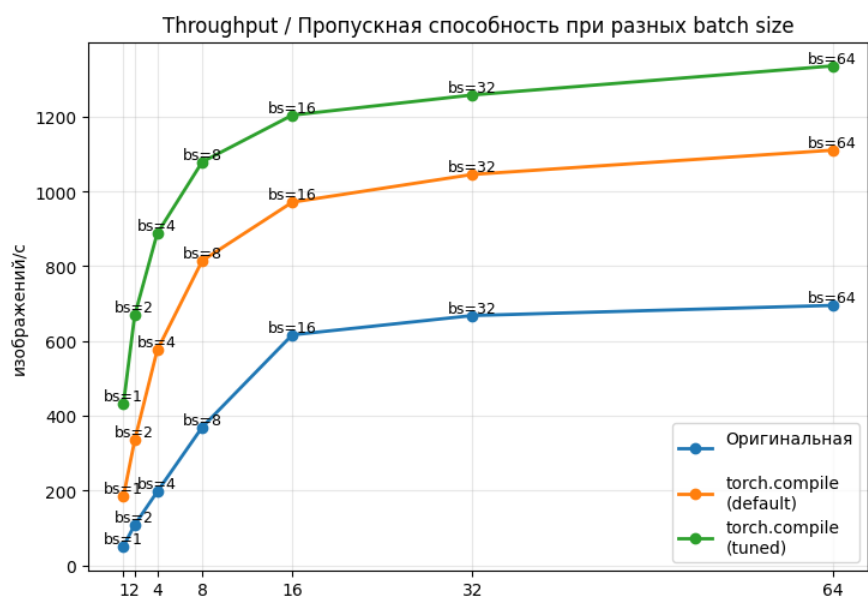


Рис. 18: График изменения пропускной способности для модели после прунинга и компиляции

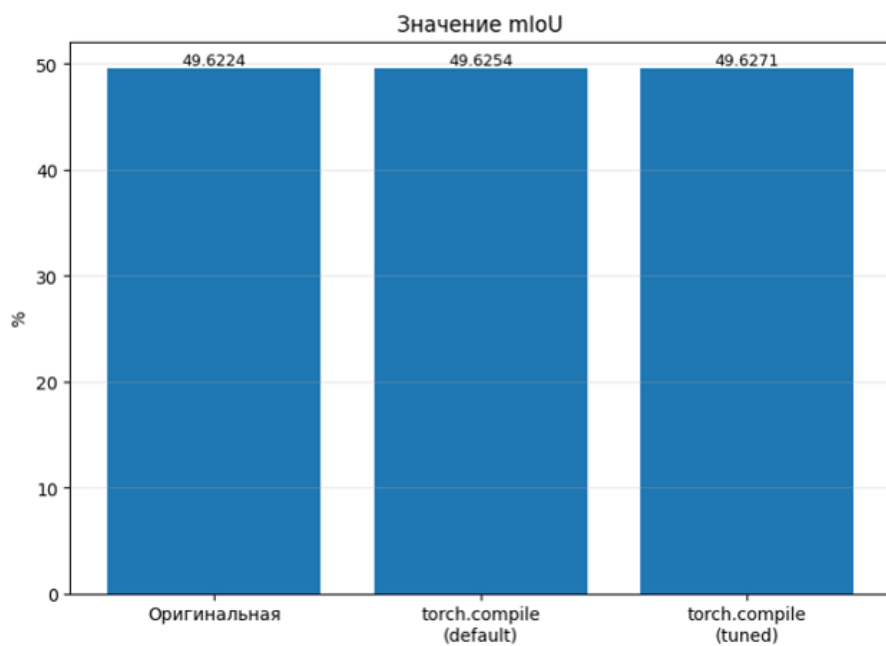


Рис. 19: График изменения качества для модели после прунинга и компиляции

7 Результаты

В итоговой таблице представлены группы экспериментов по измерениям end-to-end latency и throughput для всех конфигураций проекта. Throughput считался для batch size = 16.

Таблица 3: Группы планируемых экспериментов

Конфигурация	Latency (ms)	Throughput (img/s)	IoU (%)
<i>Baseline</i>			
PyTorch fp16 (baseline)	19.128	51.03	48.19
<i>Компильторные оптимизации</i>			
PyTorch torch.compile (mode="default")	5.825	175.54	48.19
PyTorch torch.compile (mode="max-autotune")	2.528	387.93	48.19
TVM (Relax/TIR)	26.708	37.44	45.25
<i>Aware-training оптимизация</i>			
LSQ - conv2d - квантизация	25.7	400	0.87
LSQ - triton - квантизация	40.0	150	47.5
Спарсификация	19.6	600	46.0
<i>Комбинированные методы</i>			
torch.compile(mode="default") + triton-lsq-квантизация	17.7	205	47.5
torch.compile(mode="max-autotune") + triton-lsq-квантизация	17.3	205	47.5
torch.compile(mode="default") + conv2d-lsq-квантизация	5.2	870	0.87
torch.compile(mode="max-autotune") + conv2d-lsq-квантизация	4.3	880	0.87
torch.compile(mode="default") + прунинг	6.052	1000	49.62
torch.compile(mode="max-autotune") + прунинг	2.3	1300	49.62